Application of Static Analysis (1)

CSE552 Program Analysis — Lecture 12

Jaemin Hong

C-to-Rust Translation

- Legacy system programs developed in C suffer from various security vulnerabilities because C does not ensure safety at language level
- Rust ensures memory/thread safety by detecting critical bugs at compile time through type checking
- C-to-Rust translation can improve the reliability of legacy systems by discovering previously unknown bugs and preventing new bugs in advance

Challenges in C-to-Rust Translation

- Rust requires more information to be explicitly stated in source code, compared to C, so that the compiler can utilize the information
- To translate C to Rust, such information should be inferred
- Static analysis can identify necessary information

Static Analysis for C-to-Rust Translation

- Each language feature of Rust requires distinct kinds of information
- Static analysis specialized for each language feature is required
  - Locks[1]
  - Output parameters[2]
  - Unions[3]
  - I/O[4]

---

[1] Concrat: An automatic C-to-Rust lock API translator for concurrent programs (Hong and Ryu, 2023)

[2] Don't write, but return: Replacing output parameters with algebraic data types in C-to-Rust translation (Hong and Ryu, 2024)

[3] To tag, or not to tag: Translating C's unions to Rust's tagged unions (Hong and Ryu, 2024)

[4] Forcrat: Automatic I/O API translation from C to Rust via origin and capability analysis (Hong and Ryu, 2025)

Locks in C

```
pthread_mutex_t m = ...;
int n = 0;

pthread_mutex_lock(&m);
n = n + 1;
pthread_mutex_unlock(&m);
```

- Each shared data is protected by a lock
- A thread acquires and releases the lock before and after accessing the data

Problem with Locks in C

- Implicit data-lock relation (which lock protects which data)
- Implicit flow-lock relation (which locks is held at which program point)
- The compiler does not detect programmers' mistake

Potential Data Race (1)

```
pthread_mutex_t m1 = ..., m2 = ...;
int n1 = 0, n2 = 0;

pthread_mutex_lock(&m1);
n2 = n2 + 1;
pthread_mutex_unlock(&m1);
```

- Programmers may acquire the wrong lock

Potential Data Race (2)

```
pthread_mutex_t m = ...;
int n = 0;

pthread_mutex_lock(&m);
...
pthread_mutex_unlock(&m);
n = n + 1;
```

- Programmers may acquire or release the lock at the wrong timing

Locks in Rust

```
let mut m = Mutex::new(0);

let mut guard = m.lock().unwrap();
*guard = *guard + 1;
drop(guard);
```

- A lock is a protected container for shared data
  - Rust lock = C lock + data
- A guard is a special pointer to the shared data
  - A guard is constructed only by acquiring the lock
  - When a guard is destructed, the lock is released

Advantages of Rust Locks

- Explicit data-lock relation
- Explicit flow-lock relation
- The compiler can detect programmers' mistake

Data Race Prevented

```
let mut m = Mutex::new(0);

let mut guard = m.lock().unwrap();
...
drop(guard);
*guard = *guard + 1;
```

- The compiler rejects the program because guard is not owned in the last line

Challenges in Translating Locks — Data-Lock Relations

```
pthread_mutex_t a = ..., b = ...;
int x = 0;
float y = 0.0f;
double z = 0.0;
```

- The data-lock relations must be decided

Challenges in Translating Locks — Flow-Lock Relations

```
void inc() {
  n = n + 1;
}

pthread_mutex_lock(&m);
inc();
pthread_mutex_unlock(&m);
```

```
fn inc(mut g: MutexGuard<i32
    >) -> MutexGuard<i32> {
  *g = *g + 1;
  g
}

let mut guard = m.lock().
    unwrap();
guard = inc(guard);
drop(guard);
```

- The flow-lock relations must be decided
  - Which functions should take guards
  - Which functions should return guards

Static Analysis for Lock Translation

- Unsound analysis
  - Do not consider pointer aliasing
  - When locks are aliased, the results can be unsound
- Bottom-up modular analysis
  - Each function is analyzed once to create a function summary
  - A callee is analyzed before its caller is analyzed
  - Analyzing a caller requires the callees' function summaries
- Two analyses per function
  - First, live guard analysis
  - Then, available guard analysis

Live Guard Analysis — Definition

- Goal: to decide locks that may be held at the function entry
- Similar to live variable analysis (backward, may)
- State $= (\mathcal{P}(\mathit{LockVar}), \subseteq)$
  - Locks to be released in the future
- $\mathit{JOIN}(v) = \bigcup_{u \in \mathit{succ}(v)} [\![u]\!]$
- `return`: $[\![v]\!] = \emptyset$
- `pthread_mutex_unlock(x)`: $[\![v]\!] = \mathit{JOIN}(v) \cup \{x\}$
- `pthread_mutex_lock(x)`: $[\![v]\!] = \mathit{JOIN}(v) \setminus \{x\}$

Live Guard Analysis — Examples

Example 1

```
void foo() {
 // { m }
 pthread_mutex_unlock(&m);
 // {}
}
```

Example 2

```
void foo() {
 // {}
 pthread_mutex_lock(&m);
 // { m }
 n = n + 1;
 // { m }
 pthread_mutex_unlock(&m);
 // {}
}
```

Available Guard Analysis — Definition

- Goal: to decide locks that must be held at the function return
- Similar to available expression analysis (forward, must)
- State $= (\mathcal{P}(\mathit{LockVar}), \supseteq)$
  - Locks whose guards are already available
- $\mathit{JOIN}(v) = \bigcap_{u \in \mathit{pred}(v)} [\![u]\!]$
- entry: $[\![v]\!] =$ the result of live guard analysis
- `pthread_mutex_lock(x)`: $[\![v]\!] = \mathit{JOIN}(v) \cup \{x\}$
- `pthread_mutex_unlock(x)`: $[\![v]\!] = \mathit{JOIN}(v) \setminus \{x\}$

Available Guard Analysis — Examples 1 and 2

Example 1

```
void foo() {
 // {}
 pthread_mutex_lock(&m);
 // { m }
}
```

Example 2

```
void foo() {
 // {}
 pthread_mutex_lock(&m);
 // { m }
 n = n + 1;
 // { m }
 pthread_mutex_unlock(&m);
 // {}
}
```

Available Guard Analysis — Example 3 Code

```
void foo() {
  if (...) {
    pthread_mutex_unlock(&m);
    ...
    pthread_mutex_lock(&m);
  } else {
    ...
  }
}
```

- Only when a certain condition is satisfied, temporarily releases the lock and then acquires it again

Available Guard Analysis — Example 3 Analysis

Live guard analysis

```
void foo() {
 // { m }
 if (...) {
  // { m }
  pthread_mutex_unlock(&m);
  // {}
  ...
  // {}
  pthread_mutex_lock(&m);
  // {}
 } else {
  // {}
  ...
  // {}
 }
 // {}
}
```

Available guard analysis

```
void foo() {
 // { m }
 if (...) {
  // { m }
  pthread_mutex_unlock(&m);
  // {}
  ...
  // {}
  pthread_mutex_lock(&m);
  // { m }
 } else {
  // { m }
  ...
  // { m }
 }
 // { m }
}
```

Analyzing Function Calls — Function Summary

- Function summary: locks held at the entry and return

```
void lock() {
 pthread_mutex_lock(&m);
}
```

Entry: $\{\}$, Return: $\{m\}$

```
void unlock() {
 pthread_mutex_unlock(&m);
}
```

Entry: $\{m\}$, Return: $\{\}$

Analyzing Function Calls — Constraint Rules

Live guard analysis

- `f()`: $[\![v]\!] = (\mathit{JOIN}(v) \setminus \{\text{locks held at } f\text{'s return}\}) \cup \{\text{locks held at } f\text{'s entry}\}$

Available guard analysis

- `f()`: $[\![v]\!] = (\mathit{JOIN}(v) \setminus \{\text{locks held at } f\text{'s entry}\}) \cup \{\text{locks held at } f\text{'s return}\}$

Analyzing Function Calls — Example

```
void foo() {
  // {}
  lock();
  // { m }
  n = n + 1;
  // { m }
  unlock();
  // {}
}
```

Analyzing Recursive Functions

- When a function is recursive, its own summary is not available yet
- Solution: initially analyze the function with a bottom summary (entry: $\{\}$, return: *LockVar*) and iterate until a fixed point is reached

Recursive Example 1 — Code

```
void foo(int n) {
  if (n <= 0) {
    pthread_mutex_unlock(&m);
  } else {
    foo(n - 1);
  }
}
```

- The function will eventually release the lock after some recursive calls

Recursive Example 1 — Iteration 1

Initial summary: entry $= \{\}$, return $= \mathit{LockVar}$

Live guard analysis

```
void foo(int n) {
 // { m }
 if (n <= 0) {
  // { m }
  pthread_mutex_unlock(&m);
  // {}
 } else {
  // {}
  foo(n - 1);
  // {}
 }
 // {}
}
```

Available guard analysis

```
void foo(int n) {
 // { m }
 if (n <= 0) {
  // { m }
  pthread_mutex_unlock(&m);
  // {}
 } else {
  // { m }
  foo(n - 1);
  // LockVar
 }
 // {}
}
```

Updated summary: entry $= \{m\}$, return $= \{\}$

Recursive Example 1 — Iteration 2

Live guard analysis

```
void foo(int n) {
 // { m }
 if (n <= 0) {
  // { m }
  pthread_mutex_unlock(&m);
  // {}
 } else {
  // { m }
  foo(n - 1);
  // {}
 }
 // {}
}
```

Available guard analysis

```
void foo(int n) {
 // { m }
 if (n <= 0) {
  // { m }
  pthread_mutex_unlock(&m);
  // {}
 } else {
  // { m }
  foo(n - 1);
  // {}
 }
 // {}
}
```

Fixed point: entry $= \{m\}$, return $= \{\}$

Recursive Example 2 — Code

```
void foo(int n) {
  if (n <= 0) {
    pthread_mutex_lock(&m);
  } else {
    foo(n - 1);
  }
}
```

- The function will eventually acquire the lock after some recursive calls

Recursive Example 2 — Iteration 1

Initial summary: entry $= \{\}$, return $= \mathit{LockVar}$

Live guard analysis

```
void foo(int n) {
 // {}
 if (n <= 0) {
  // {}
  pthread_mutex_lock(&m);
  // {}
 } else {
  // {}
  foo(n - 1);
  // {}
 }
 // {}
}
```

Available guard analysis

```
void foo(int n) {
 // {}
 if (n <= 0) {
  // {}
  pthread_mutex_lock(&m);
  // { m }
 } else {
  // {}
  foo(n - 1);
  // LockVar
 }
 // { m }
}
```

Updated summary: entry $= \{\}$, return $= \{m\}$

Recursive Example 2 — Iteration 2

Live guard analysis

```
void foo(int n) {
 // {}
 if (n <= 0) {
  // {}
  pthread_mutex_lock(&m);
  // {}
 } else {
  // {}
  foo(n - 1);
  // {}
 }
 // {}
}
```

Available guard analysis

```
void foo(int n) {
 // {}
 if (n <= 0) {
  // {}
  pthread_mutex_lock(&m);
  // { m }
 } else {
  // {}
  foo(n - 1);
  // { m }
 }
 // { m }
}
```

Fixed point: entry $= \{\}$, return $= \{m\}$

Mutual Recursion

- Start with a bottom summary for each function in the cycle
- Analyze each function with the same summaries and update their summaries
- Finish when all summaries reach their fixed point

Propagating Call-Site Information

- Locks held at call sites should be considered in callees
- A single top-down pass after the bottom-up analysis

Propagating Call-Site Information — Example

```
void inc() {
  // {}
  n = n + 1;
  // {}
}

void foo() {
  // {}
  pthread_mutex_lock(&m);
  // { m }
  inc();
  // { m }
  pthread_mutex_unlock(&m);
  // {}
}
```

$\Rightarrow$

```
void inc() {
  // { m }
  n = n + 1;
  // { m }
}

void foo() {
  // {}
  pthread_mutex_lock(&m);
  // { m }
  inc();
  // { m }
  pthread_mutex_unlock(&m);
  // {}
}
```

`m` is held at every call site of `inc`

Deciding Data-Lock Relations

- If data is accessed only when a certain lock is held, it is considered to be protected by the lock

Translation Rules

- When `m` protects `n`:
  - Merge `m` and `n` into a single Rust lock
  - `pthread_mutex_lock`: call `lock` and assigns to `m_guard`
  - `pthread_mutex_unlock`: drop `m_guard`
  - `n`: `*m_guard`
- When `m` is held at the entry:
  - Take `m_guard` as a parameter
- When `m` is held at the return:
  - Return `m_guard`

Translation Example

```
pthread_mutex_t m = ...;
int n = 0;

void inc() {
  n = n + 1;
}

pthread_mutex_lock(&m);
inc();
pthread_mutex_unlock(&m);
```

```
let mut m: Mutex<i32> =
    Mutex::new(0);

fn inc(mut g: MutexGuard<i32
    >) -> MutexGuard<i32> {
  *g = *g + 1;
  g
}

let mut guard = m.lock().
    unwrap();
guard = inc(guard);
drop(guard);
```

Summary

- C-to-Rust translation can improve legacy systems' reliability, but Rust demands information to be inferred from C code
- Lock translation must recover both data-lock and flow-lock relations that are implicit in C
- Live guard analysis (backward, may) computes locks held at function entry; available guard analysis (forward, must) computes locks held at return
- Function summaries enable bottom-up modular analysis; recursive functions are handled by fixed-point iteration from a bottom summary